Heart Disease Prediction Using Machine Learning

Dataset: `heart.csv`

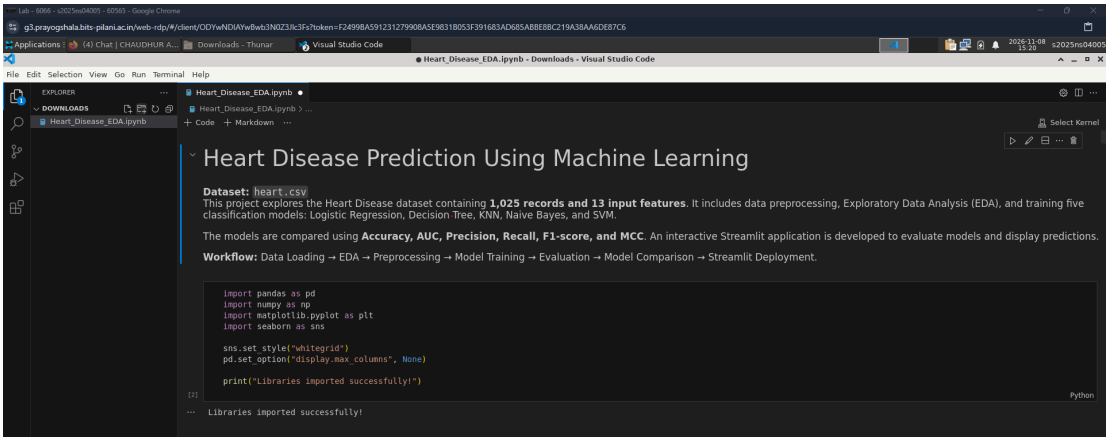
This project explores the Heart Disease dataset containing **1,025 records and 13 input features**. It includes data preprocessing, Exploratory Data Analysis (EDA), and training five classification models: Logistic Regression, Decision Tree, KNN, Naive Bayes, and SVM.

The models are compared using **Accuracy, AUC, Precision, Recall, F1-score, and MCC**. An interactive Streamlit application is developed to evaluate models and display predictions.

Workflow: Data Loading → EDA → Preprocessing → Model Training → Evaluation → Model Comparison → Streamlit Deployment.

Git Hub Repository link:
https://github.com/amanchaudhur/Heart_Disease_ML_Assignment

Streamlit App link: https://ga7v3aub5gxcxdr5mxfsxz.streamlit.app/



1. Import Libraries

In [4]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")
pd.set_option("display.max_columns", None)

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

2. Load the dataset

In [5]:
```
from pathlib import Path

data_path = Path(r"C:\Users\amanc\OneDrive\Desktop\Heart_Disease_ML_Assignment\h


df = pd.read_csv(data_path)
print("Dataset loaded successfully!")
df.head()
```

```
Dataset loaded successfully!
```

Out[5]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	57	1	2	150	168	0	1	174	0	1.6226	2	0	2
1	49	1	2	118	149	0	0	126	0	0.8058	2	3	2
2	62	0	0	160	164	0	0	145	0	6.2146	0	3	3
3	70	1	2	160	269	0	1	112	1	2.9000	1	1	3
4	63	0	1	140	195	0	1	179	0	0.0320	2	2	2

3. Dataset dimensions and column names

In [6]:
```
print("Number of records:", df.shape[0])
print("Number of columns:", df.shape[1])
print("Dataset dimensions:", df.shape)

print("\\nColumn names:")
print(df.columns.tolist())
```

```
Number of records: 1025
Number of columns: 14
Dataset dimensions: (1025, 14)
\nColumn names:
['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'o
ldpeak', 'slope', 'ca', 'thal', 'target']
```

4. Data types and non-null counts

In [7]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   age       1025 non-null   int64  
 1   sex       1025 non-null   int64  
 2   cp        1025 non-null   int64  
 3   trestbps  1025 non-null   int64  
 4   chol      1025 non-null   int64  
 5   fbs       1025 non-null   int64  
 6   restecg   1025 non-null   int64  
 7   thalach   1025 non-null   int64  
 8   exang     1025 non-null   int64  
 9   oldpeak   1025 non-null   float64
 10  slope     1025 non-null   int64  
 11  ca        1025 non-null   int64  
 12  thal      1025 non-null   int64  
 13  target    1025 non-null   int64  
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
```

5. Missing-value analysis

In [8]:
```
missing_values = df.isnull().sum()
display(missing_values.to_frame(name="Missing values"))

print("Total missing values:", int(missing_values.sum()))
```

	Missing values
age	0
sex	0
cp	0
trestbps	0
chol	0
fbs	0
restecg	0
thalach	0
exang	0
oldpeak	0
slope	0
ca	0
thal	0
target	0

```
Total missing values: 0
```

In [9]:
```
plt.figure(figsize=(10, 4))
sns.heatmap(df.isnull(), cbar=False, yticklabels=False, cmap="viridis")
plt.title("Missing Values Overview")
plt.xlabel("Columns")
plt.tight_layout()
plt.show()
```



6. Duplicate-record analysis

In [10]:
```
duplicate_count = int(df.duplicated().sum())
unique_rows = int(len(df.drop_duplicates()))

print("Total records:", len(df))
print("Exact duplicate rows after the first occurrence:", duplicate_count)
print("Unique rows:", unique_rows)
print("Percentage of rows flagged as duplicates:", round(duplicate_count / len(d
```

```
Total records: 1025
Exact duplicate rows after the first occurrence: 205
Unique rows: 820
Percentage of rows flagged as duplicates: 20.0 %
```

7. Statistical summary

In [11]:
```
display(df.describe().T)
```

	count	mean	std	min	25%	50%	75%	max
age	1025.0	54.628293	9.220147	29.0	48.0000	56.0	62.0	77.0000
sex	1025.0	0.687805	0.463615	0.0	0.0000	1.0	1.0	1.0000
cp	1025.0	0.947317	1.012208	0.0	0.0000	1.0	2.0	3.0000
trestbps	1025.0	131.482927	17.719713	94.0	120.0000	130.0	140.0	200.0000
chol	1025.0	246.952195	52.439536	126.0	211.0000	243.0	278.0	564.0000
fbs	1025.0	0.150244	0.357485	0.0	0.0000	0.0	0.0	1.0000
restecg	1025.0	0.491707	0.524944	0.0	0.0000	0.0	1.0	2.0000
thalach	1025.0	148.402927	23.100337	71.0	132.0000	152.0	165.0	202.0000
exang	1025.0	0.341463	0.474432	0.0	0.0000	0.0	1.0	1.0000
oldpeak	1025.0	1.084124	1.194273	0.0	0.0315	0.8	1.8	6.2256
slope	1025.0	1.378537	0.616448	0.0	1.0000	1.0	2.0	2.0000
ca	1025.0	0.766829	1.048284	0.0	0.0000	0.0	1.0	4.0000
thal	1025.0	2.311220	0.623217	0.0	2.0000	2.0	3.0	3.0000
target	1025.0	0.504390	0.500225	0.0	0.0000	1.0	1.0	1.0000

8. Target-variable distribution

In [12]:
```
target_col = "target"

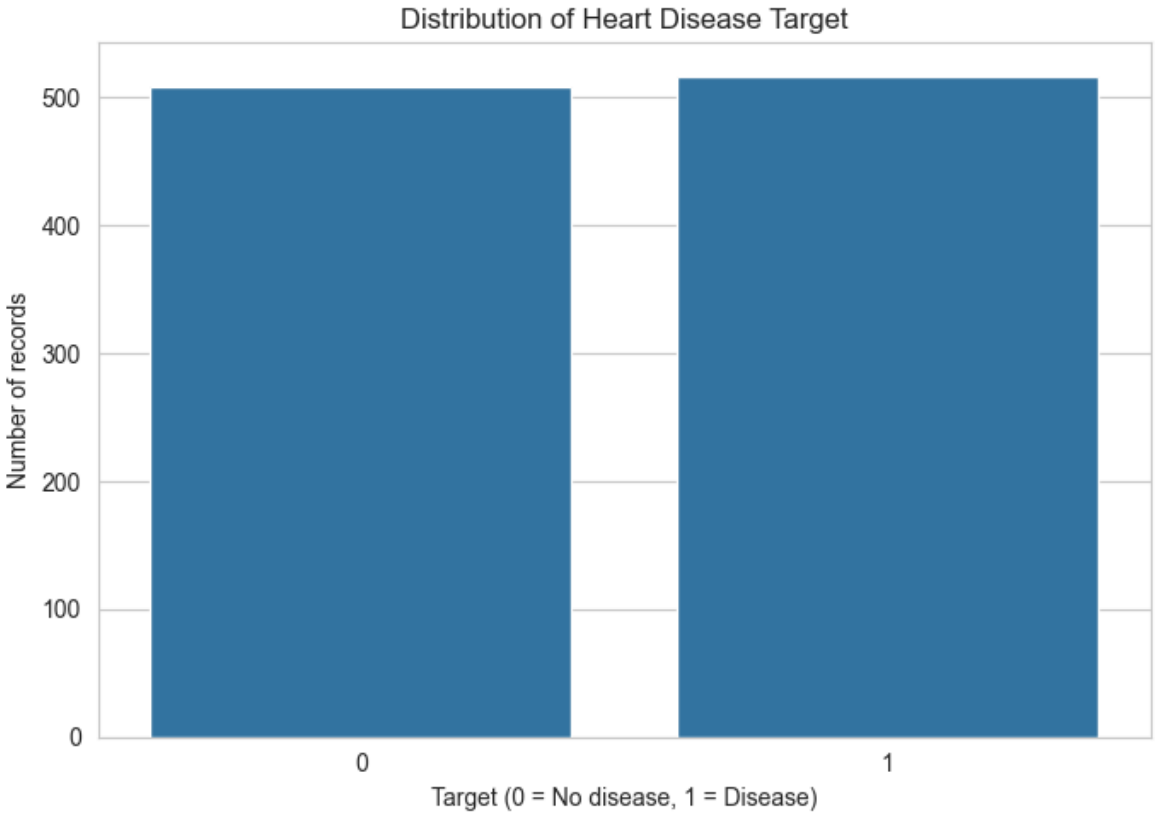
if target_col not in df.columns:
    raise ValueError("Expected target column 'target' was not found. Check df.co

target_counts = df[target_col].value_counts().sort_index()
target_percentages = (df[target_col].value_counts(normalize=True).sort_index() *

display(pd.DataFrame({
    "Count": target_counts,
    "Percentage": target_percentages
}))

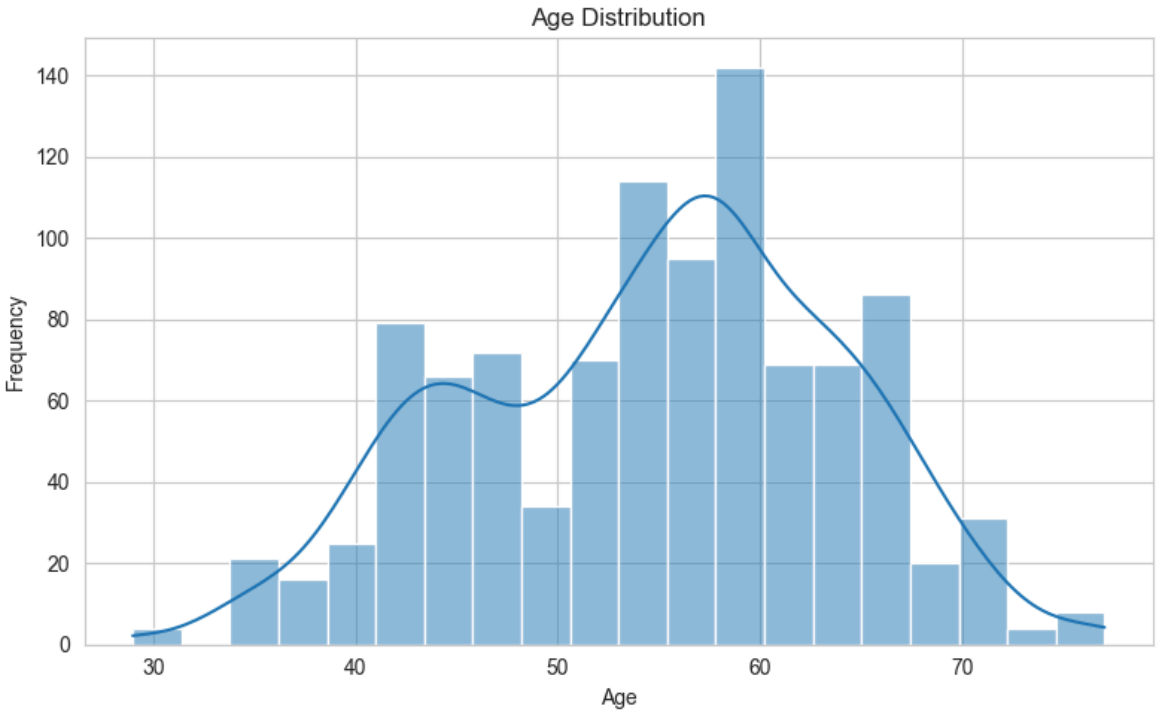
plt.figure(figsize=(7, 5))
ax = sns.countplot(data=df, x=target_col)
ax.set_title("Distribution of Heart Disease Target")
ax.set_xlabel("Target (0 = No disease, 1 = Disease)")
ax.set_ylabel("Number of records")
plt.tight_layout()
plt.show()
```

target	Count	Percentage
0	508	49.56
1	517	50.44



9. Age distribution

In [13]:
```
if "age" in df.columns:
    plt.figure(figsize=(8, 5))
    sns.histplot(data=df, x="age", bins=20, kde=True)
    plt.title("Age Distribution")
    plt.xlabel("Age")
    plt.ylabel("Frequency")
    plt.tight_layout()
    plt.show()
else:
    print("Column 'age' was not found.")
```



10. Distributions of selected numerical features

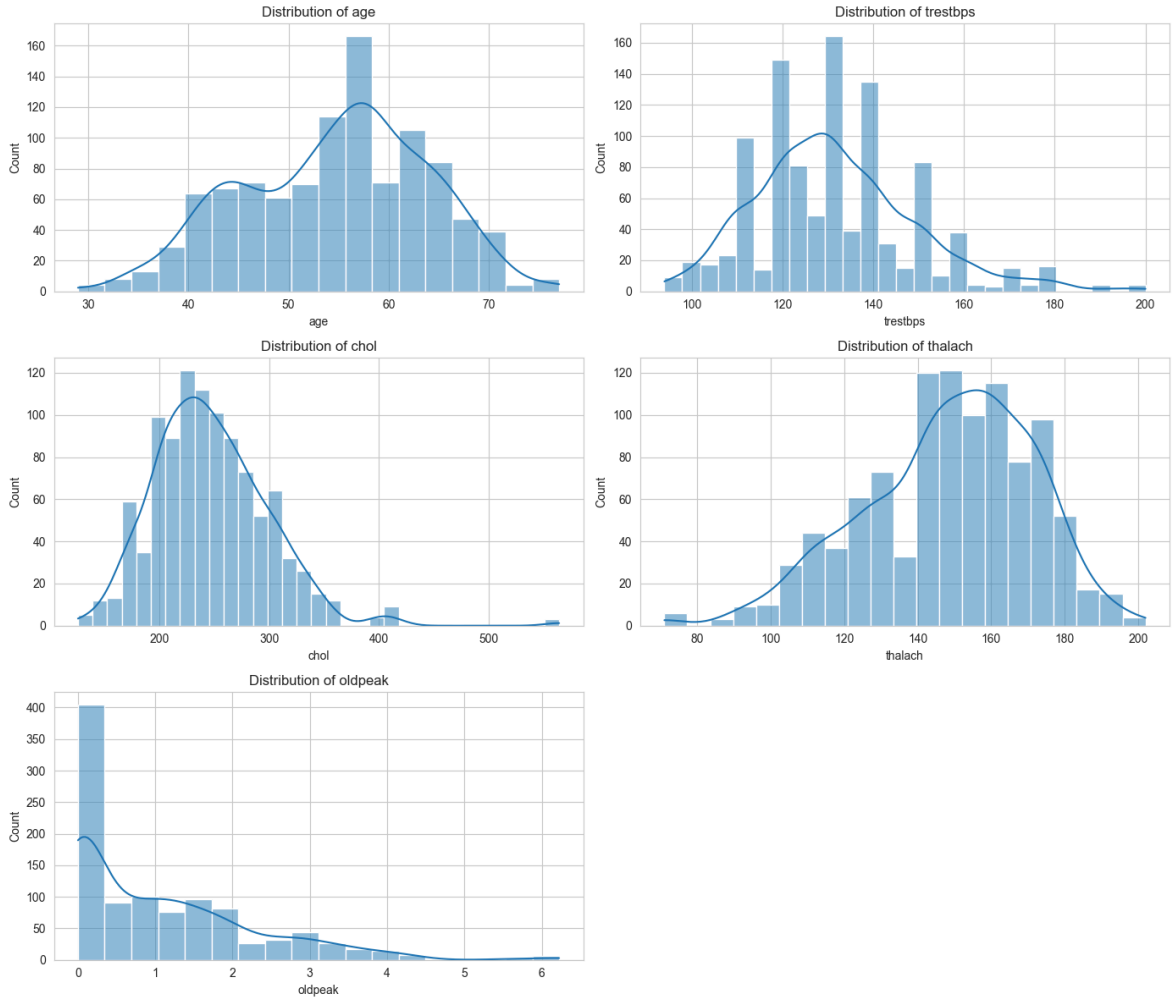
In [14]:
```
numerical_features = [
    col for col in ["age", "trestbps", "chol", "thalach", "oldpeak"]
    if col in df.columns
]

if numerical_features:
    ncols = 2
    nrows = int(np.ceil(len(numerical_features) / ncols))
    fig, axes = plt.subplots(nrows, ncols, figsize=(14, 4 * nrows))
    axes = np.array(axes).reshape(-1)

    for ax, column in zip(axes, numerical_features):
        sns.histplot(data=df, x=column, kde=True, ax=ax)
        ax.set_title(f"Distribution of {column}")

    for ax in axes[len(numerical_features):]:
        ax.remove()

    plt.tight_layout()
    plt.show()
else:
    print("None of the selected numerical columns were found.")
```



11. Distributions of encoded categorical features

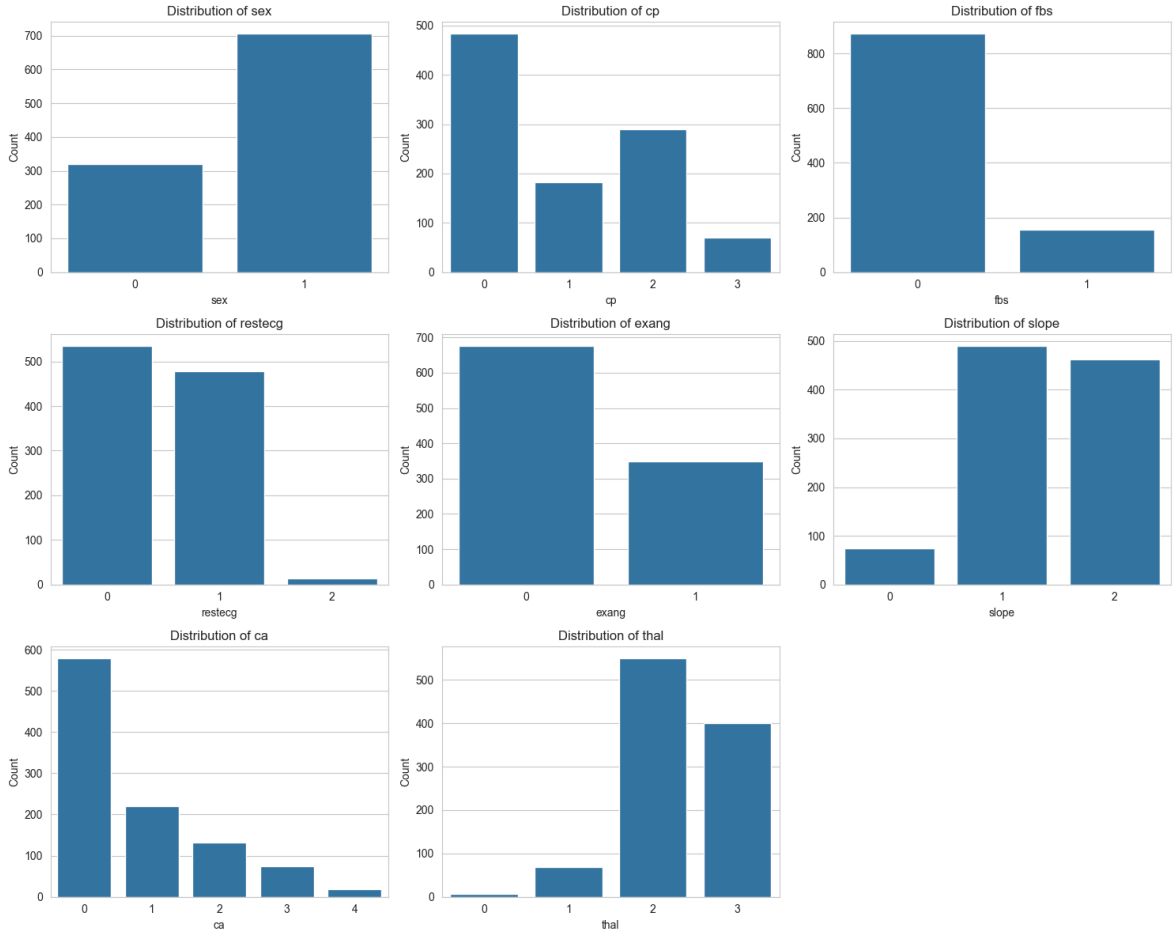
In [15]:
```
categorical_features = [
    col for col in ["sex", "cp", "fbs", "restecg", "exang", "slope", "ca", "thal
    if col in df.columns
]

if categorical_features:
    ncols = 3
    nrows = int(np.ceil(len(categorical_features) / ncols))
    fig, axes = plt.subplots(nrows, ncols, figsize=(15, 4 * nrows))
    axes = np.array(axes).reshape(-1)

    for ax, column in zip(axes, categorical_features):
        sns.countplot(data=df, x=column, ax=ax)
        ax.set_title(f"Distribution of {column}")
        ax.set_xlabel(column)
        ax.set_ylabel("Count")

    for ax in axes[len(categorical_features):]:
        ax.remove()

    plt.tight_layout()
    plt.show()
else:
    print("None of the selected categorical columns were found.")
```

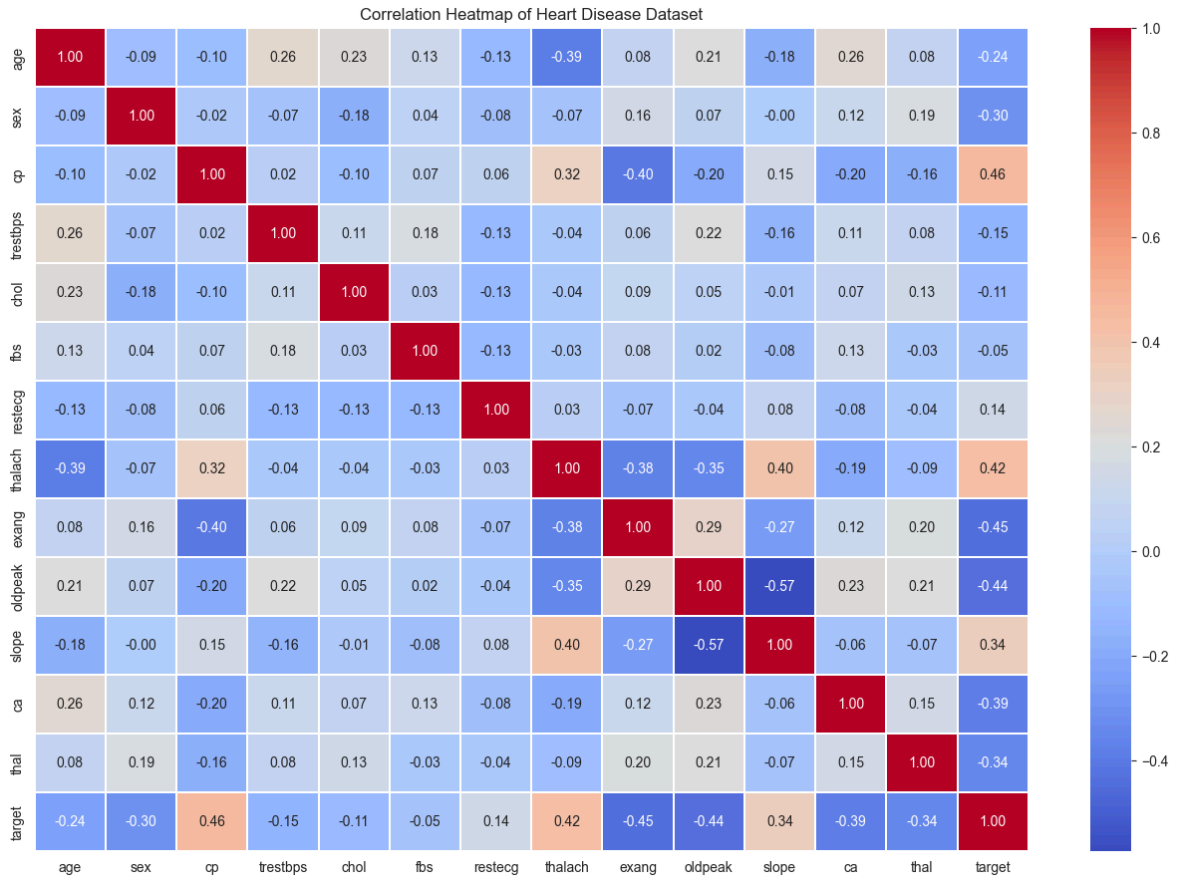


Note: These columns are numerically encoded categories. Their numeric codes should not automatically be interpreted as continuous measurements.

12. Correlation heatmap

In [16]:
```
# Correlation is calculated for numeric columns.
correlation_matrix = df.corr(numeric_only=True)

plt.figure(figsize=(13, 9))
sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.3
)
plt.title("Correlation Heatmap of Heart Disease Dataset")
plt.tight_layout()
plt.show()
```



13. Feature distributions by target class

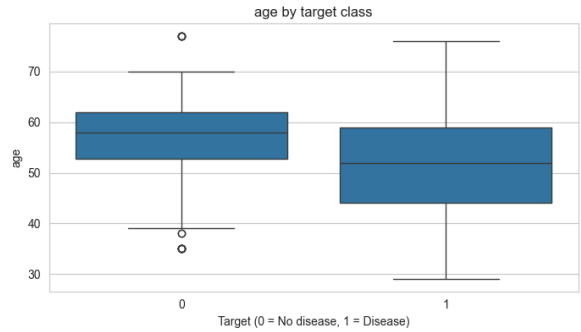
In [17]:
```
features_to_compare = [
    col for col in ["age", "thalach", "chol", "trestbps", "oldpeak"]
    if col in df.columns
]

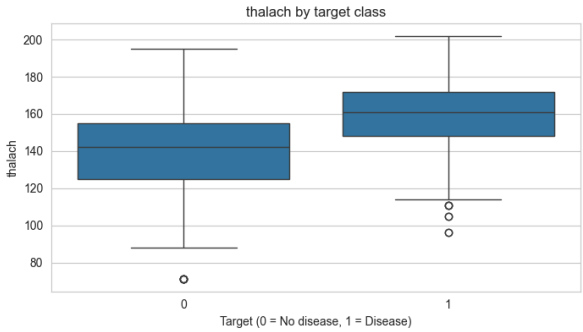
if features_to_compare:
    ncols = 2
    nrows = int(np.ceil(len(features_to_compare) / ncols))
    fig, axes = plt.subplots(nrows, ncols, figsize=(14, 4 * nrows))
    axes = np.array(axes).reshape(-1)

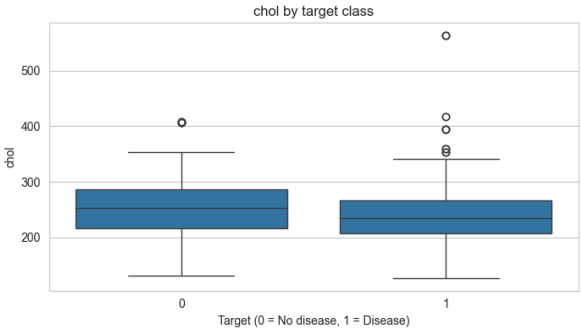
    for ax, column in zip(axes, features_to_compare):
        sns.boxplot(data=df, x=target_col, y=column, ax=ax)
        ax.set_title(f"{column} by target class")
        ax.set_xlabel("Target (0 = No disease, 1 = Disease)")

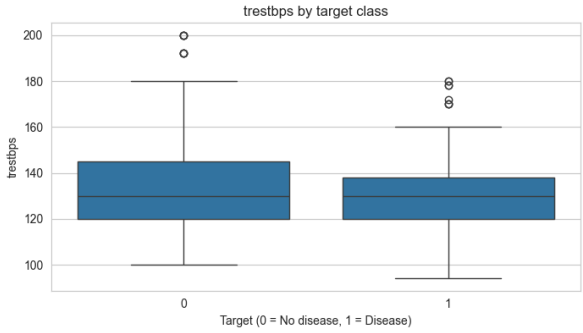
    for ax in axes[len(features_to_compare):]:
        ax.remove()

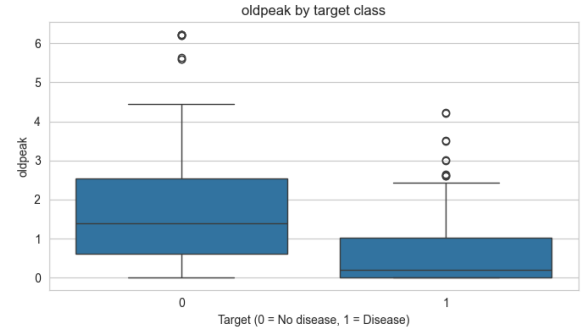
    plt.tight_layout()
    plt.show()
else:
    print("No selected comparison features were found.")
```









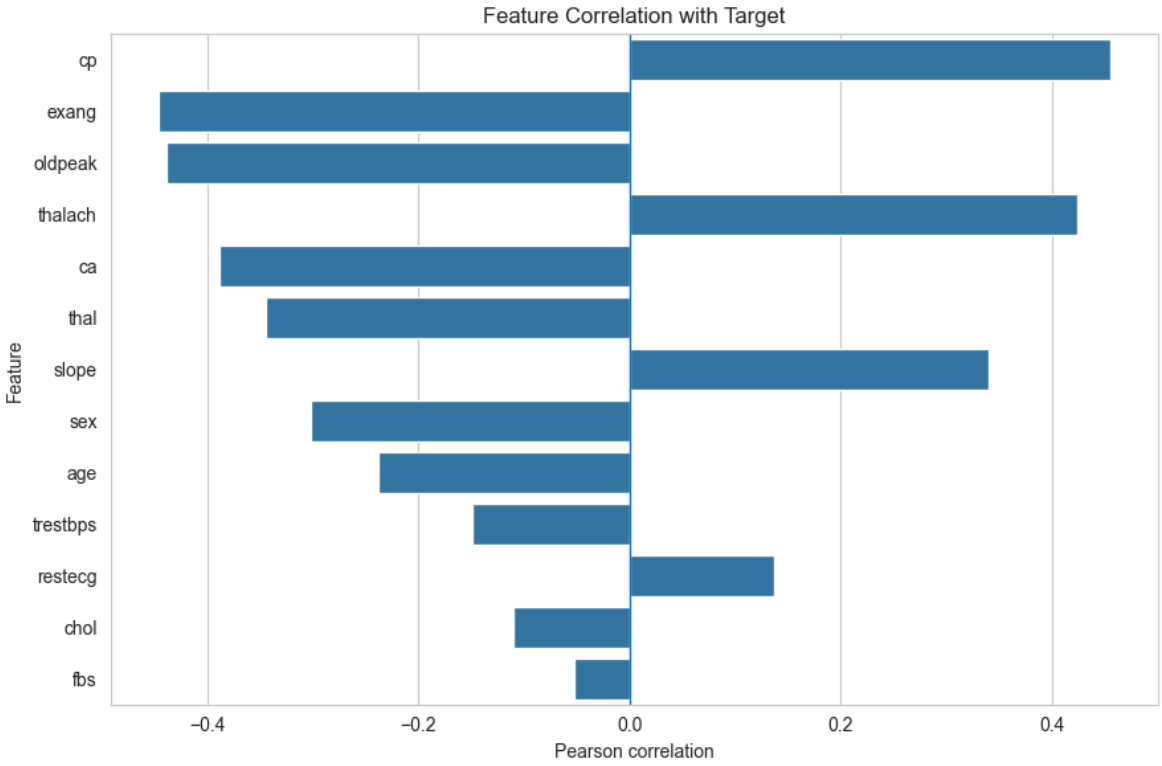


14. Feature-to-target correlations

In [18]:
```
if target_col in correlation_matrix.columns:
    target_correlations = (
        correlation_matrix[target_col]
        .drop(labels=[target_col])
        .sort_values(key=lambda values: values.abs(), ascending=False)
    )
    display(target_correlations.to_frame(name="Correlation with target"))

    plt.figure(figsize=(9, 6))
    sns.barplot(
        x=target_correlations.values,
        y=target_correlations.index
    )
    plt.axvline(0, linewidth=1)
    plt.title("Feature Correlation with Target")
    plt.xlabel("Pearson correlation")
    plt.ylabel("Feature")
    plt.tight_layout()
    plt.show()
```

	Correlation with target
cp	0.455631
exang	-0.446619
oldpeak	-0.438544
thalach	0.424391
ca	-0.388203
thal	-0.344267
slope	0.339801
sex	-0.301481
age	-0.237321
trestbps	-0.149139
restecg	0.136811
chol	-0.110282
fbs	-0.052842



15. Generate a preliminary EDA summary

In [19]:
```
print("EDA SUMMARY")
print("-" * 40)
print(f"Records: {df.shape[0]}")
print(f"Columns: {df.shape[1]}")
print(f"Input features (excluding target): {df.shape[1] - 1}")
print(f"Missing values: {int(df.isnull().sum().sum())}")
print(f"Exact duplicate rows: {int(df.duplicated().sum())}")
```

```
if target_col in df.columns:
    print("\\nTarget distribution:")
    print(df[target_col].value_counts().sort_index())

if "age" in df.columns:
    print(f"\\nMean age: {df['age'].mean():.2f}")
```

```
EDA SUMMARY
----------------------------------------
Records: 1025
Columns: 14
Input features (excluding target): 13
Missing values: 0
Exact duplicate rows: 205
\nTarget distribution:
target
0    508
1    517
Name: count, dtype: int64
\nMean age: 54.63
```

16. Exploratory Data Analysis (EDA) – Conclusion

The Exploratory Data Analysis of the Heart Disease dataset provided useful insights into the dataset's quality, distribution and relationships between different features and the target variable.

The following observations were made:

1. **Dataset Overview:** The dataset contains 1,025 records and 14 columns, consisting of 13 input features and one binary target variable (`target`). The average age is approximately 54.63 years, with ages ranging from 29 to 77 years.

2. **Missing Values:** No missing values were found in any column. Therefore, the dataset does not require missing-value imputation based on the observed data.

3. **Duplicate Records:** The dataset contains 205 exact duplicate rows (20%), leaving 820 unique records. These duplicates require special consideration to prevent data leakage during model training and evaluation.

4. **Target Distribution:** The target variable is nearly balanced, with 508 records (49.56%) belonging to class 0 and 517 records (50.44%) belonging to class 1. This balance is beneficial for comparing classification algorithms.

5. **Feature Distributions:** The numerical features show different distributions and ranges. Some features, particularly cholesterol (`chol`) and ST depression (`oldpeak`), contain relatively high values and potential outliers. Several categorical features are represented using numerical codes.

6. **Correlation Analysis:** Chest pain type (`cp` , +0.456) and maximum heart rate (`thalach` , +0.424) show notable positive correlations with the target. Exercise-induced angina (`exang` , −0.447), ST depression (`oldpeak` , −0.439) and number of

major vessels (`ca`, −0.388) show notable negative correlations. These associations suggest that several features may be useful for classification, although correlation does not establish causation.

7. **Feature Relationships:** The boxplots show differences between the target classes, particularly for maximum heart rate and ST depression. However, the distributions overlap, indicating that combining multiple features may be more informative than relying on a single feature.

Final Conclusion

Overall, the dataset is suitable for the planned machine learning classification experiments because it contains sufficient records, 13 input features, no missing values and a nearly balanced target variable.

The main data-quality concern is the presence of duplicate records. Therefore, a group-aware train-test split should be used to prevent identical feature records from appearing in both sets. Feature scaling should also be fitted only on the training data to avoid data leakage.

The EDA findings provide a foundation for training and comparing Logistic Regression, Decision Tree, K-Nearest Neighbors, Gaussian Naive Bayes and Support Vector Machine classifiers using Accuracy, AUC, Precision, Recall, F1-score and Matthews Correlation Coefficient.

17. Train-test splitting and feature scaling

We'll use an 80:20 split and group identical feature records to reduce leakage. Scaling and imputation will happen inside each model pipeline, fitted only on training data.

In [20]:
```
from sklearn.model_selection import GroupShuffleSplit
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline

X = df.drop(columns=["target"])
y = df["target"]

groups = pd.util.hash_pandas_object(
    X.reset_index(drop=True), index=False
)

splitter = GroupShuffleSplit(
    n_splits=1,
    test_size=0.20,
    random_state=42
)

train_idx, test_idx = next(splitter.split(X, y, groups))

X_train = X.iloc[train_idx]
X_test = X.iloc[test_idx]
```

```
y_train = y.iloc[train_idx]
y_test = y.iloc[test_idx]

print("Training:", X_train.shape)
print("Testing:", X_test.shape)
print("Training class counts:\n", y_train.value_counts())
print("Testing class counts:\n", y_test.value_counts())

assert set(groups.iloc[train_idx]).isdisjoint(
    set(groups.iloc[test_idx])
)
```

```
Training: (821, 13)
Testing: (204, 13)
Training class counts:
 target
1    423
0    398
Name: count, dtype: int64
Testing class counts:
 target
0    110
1     94
Name: count, dtype: int64
```

18. Train five classification models

In [21]:

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC

from sklearn.base import clone

estimators = {
    "Logistic Regression": LogisticRegression(
        max_iter=2000, random_state=42
    ),
    "Decision Tree": DecisionTreeClassifier(
        random_state=42
    ),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Naive Bayes": GaussianNB(),
    "SVM": SVC(
        kernel="rbf",
        probability=True,
        random_state=42
    )
}

models = {}

for name, estimator in estimators.items():
    pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("classifier", clone(estimator))
    ])
```

```
    pipeline.fit(X_train, y_train)
    models[name] = pipeline

    print(name, "trained successfully")
```

```
Logistic Regression trained successfully
Decision Tree trained successfully
KNN trained successfully
Naive Bayes trained successfully
SVM trained successfully
```

19. Calculate all six evaluation metrics

In [22]:
```
from sklearn.metrics import (
    accuracy_score,
    roc_auc_score,
    precision_score,
    recall_score,
    f1_score,
    matthews_corrcoef,
    confusion_matrix,
    classification_report
)

results = []

for name, model in models.items():
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "AUC": roc_auc_score(y_test, y_prob),
        "Precision": precision_score(y_test, y_pred, zero_division=0),
        "Recall": recall_score(y_test, y_pred, zero_division=0),
        "F1": f1_score(y_test, y_pred, zero_division=0),
        "MCC": matthews_corrcoef(y_test, y_pred)
    })

results_df = pd.DataFrame(results)
display(results_df.round(4))
```

	Model	Accuracy	AUC	Precision	Recall	F1	MCC
0	Logistic Regression	0.8627	0.9434	0.8173	0.9043	0.8586	0.7294
1	Decision Tree	0.9755	0.9750	0.9785	0.9681	0.9733	0.9507
2	KNN	0.8971	0.9515	0.9011	0.8723	0.8865	0.7927
3	Naive Bayes	0.8431	0.9435	0.7981	0.8830	0.8384	0.6901
4	SVM	0.9265	0.9732	0.9072	0.9362	0.9215	0.8527

20. confusion matrix and classification report

In [23]:
```
for name, model in models.items():
    y_pred = model.predict(X_test)

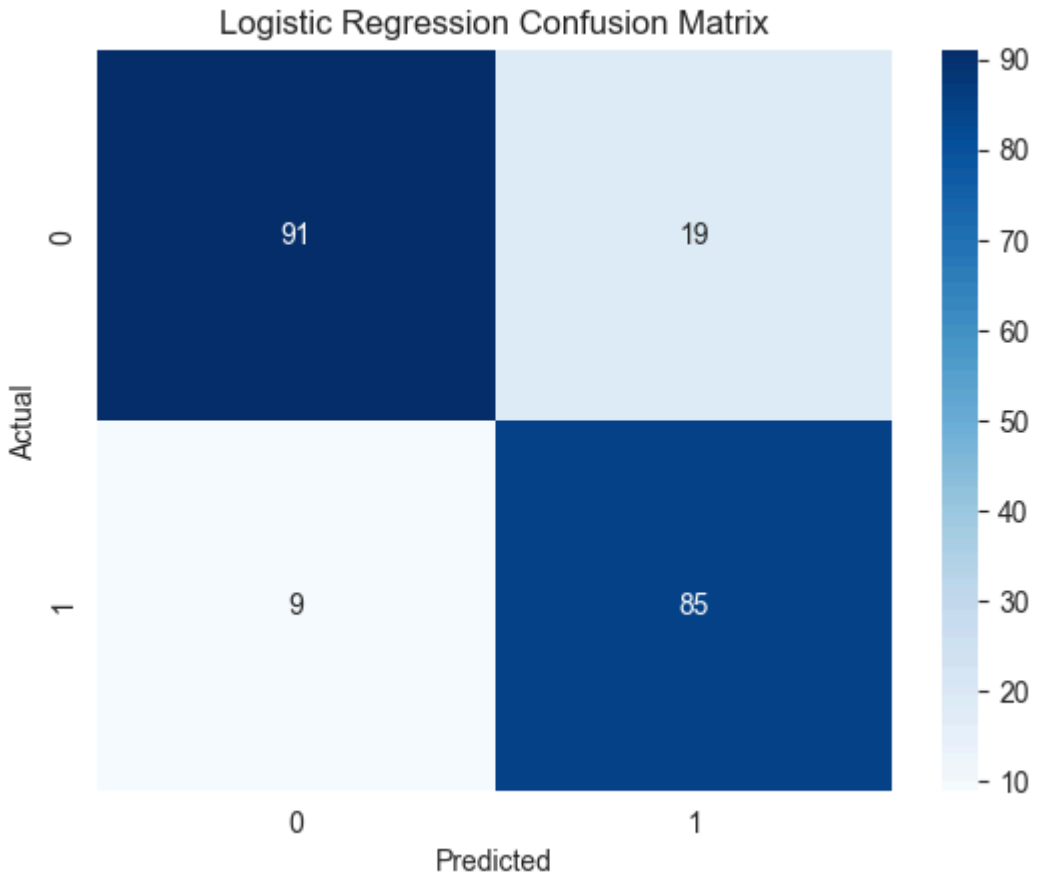
    print("\nMODEL:", name)
    print(classification_report(
        y_test, y_pred, zero_division=0
    ))

    sns.heatmap(
        confusion_matrix(y_test, y_pred, labels=[0, 1]),
        annot=True,
        fmt="d",
        cmap="Blues"
    )
    plt.title(f"{name} Confusion Matrix")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()
```

```
MODEL: Logistic Regression
              precision    recall  f1-score   support

           0       0.91      0.83      0.87       110
           1       0.82      0.90      0.86        94

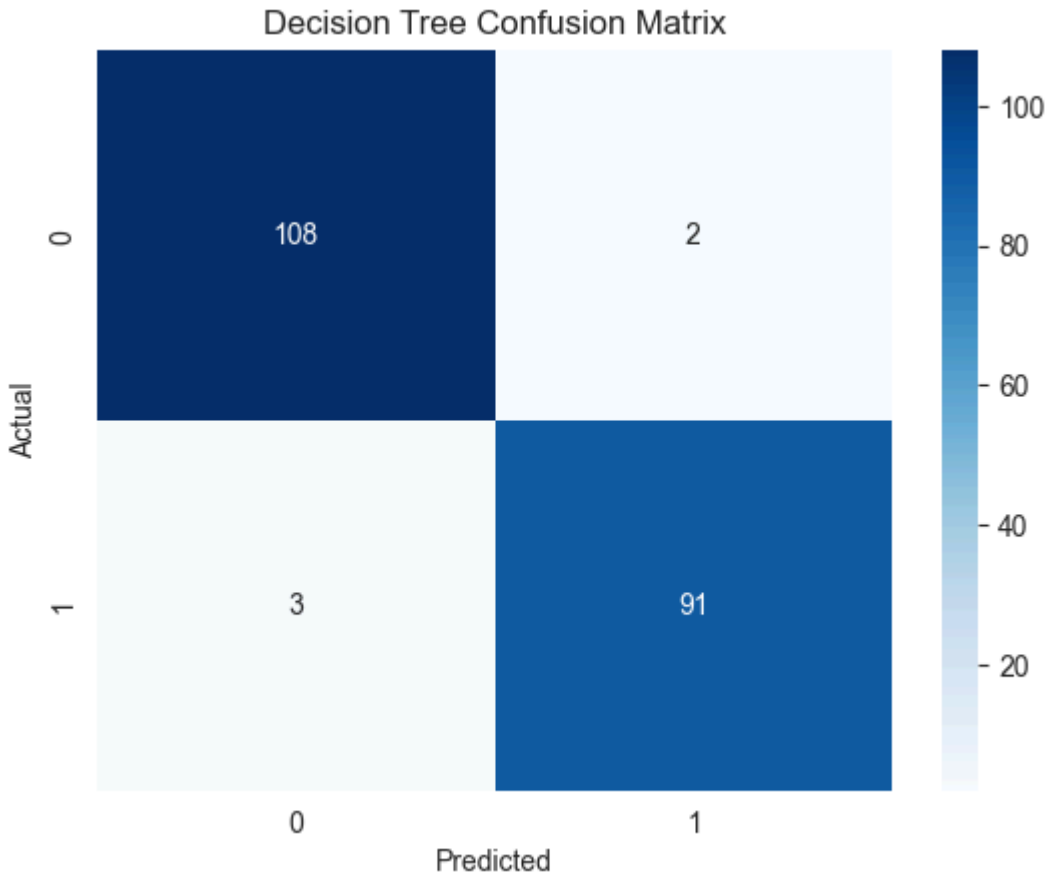
    accuracy                           0.86       204
   macro avg       0.86      0.87      0.86       204
weighted avg       0.87      0.86      0.86       204
```



```
MODEL: Decision Tree
              precision    recall  f1-score   support

           0       0.97      0.98      0.98       110
           1       0.98      0.97      0.97        94

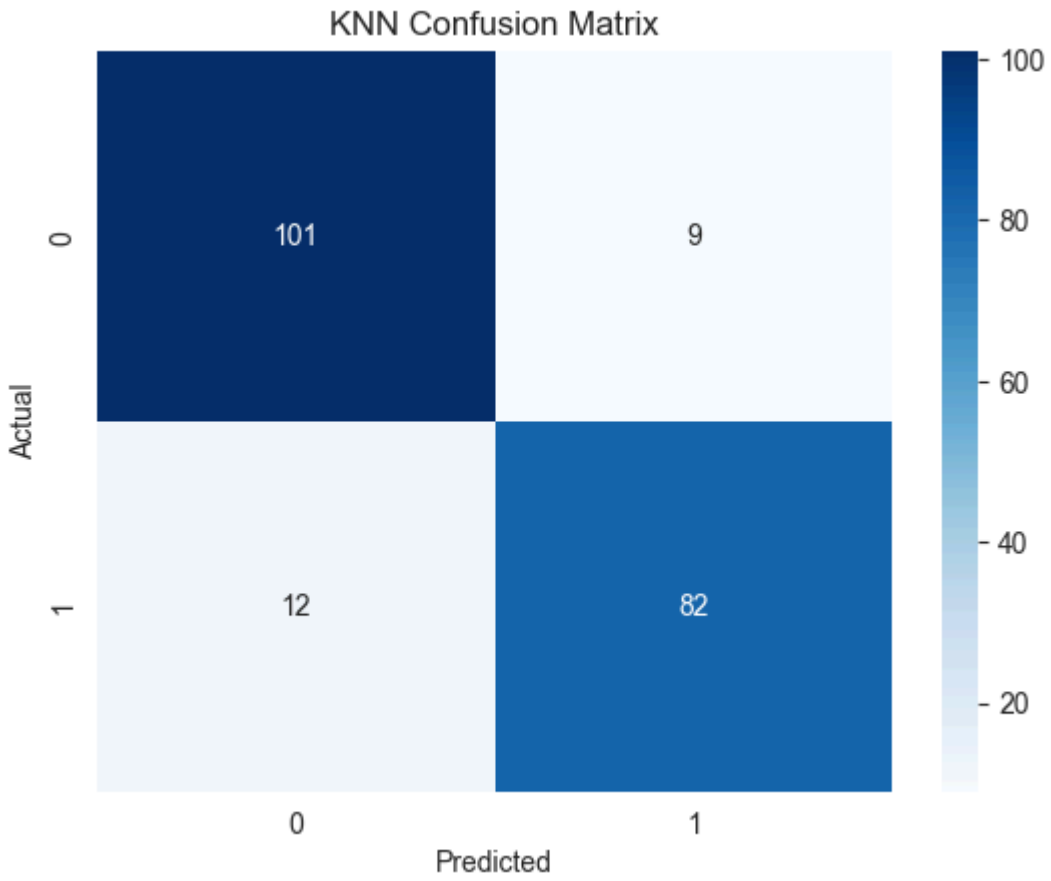
    accuracy                           0.98       204
   macro avg       0.98      0.97      0.98       204
weighted avg       0.98      0.98      0.98       204
```




```
MODEL: KNN
              precision    recall  f1-score   support

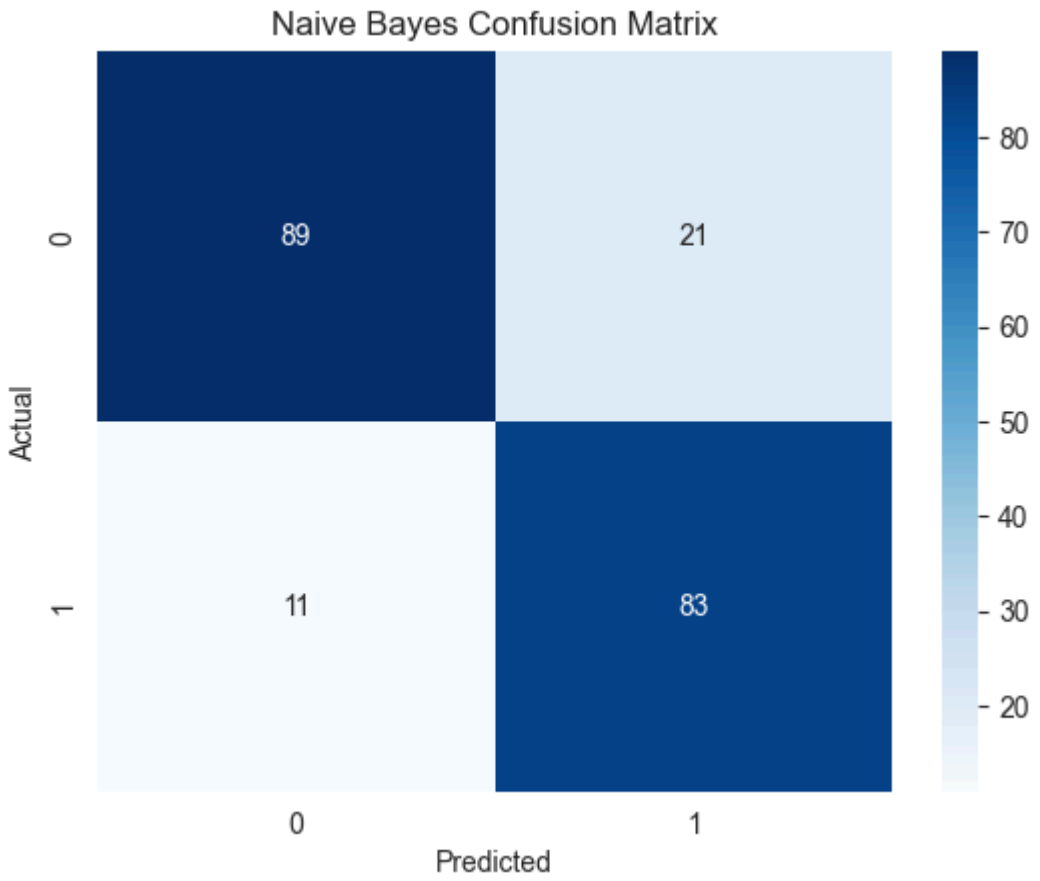
           0       0.89      0.92      0.91       110
           1       0.90      0.87      0.89        94

    accuracy                           0.90       204
   macro avg       0.90      0.90      0.90       204
weighted avg       0.90      0.90      0.90       204
```



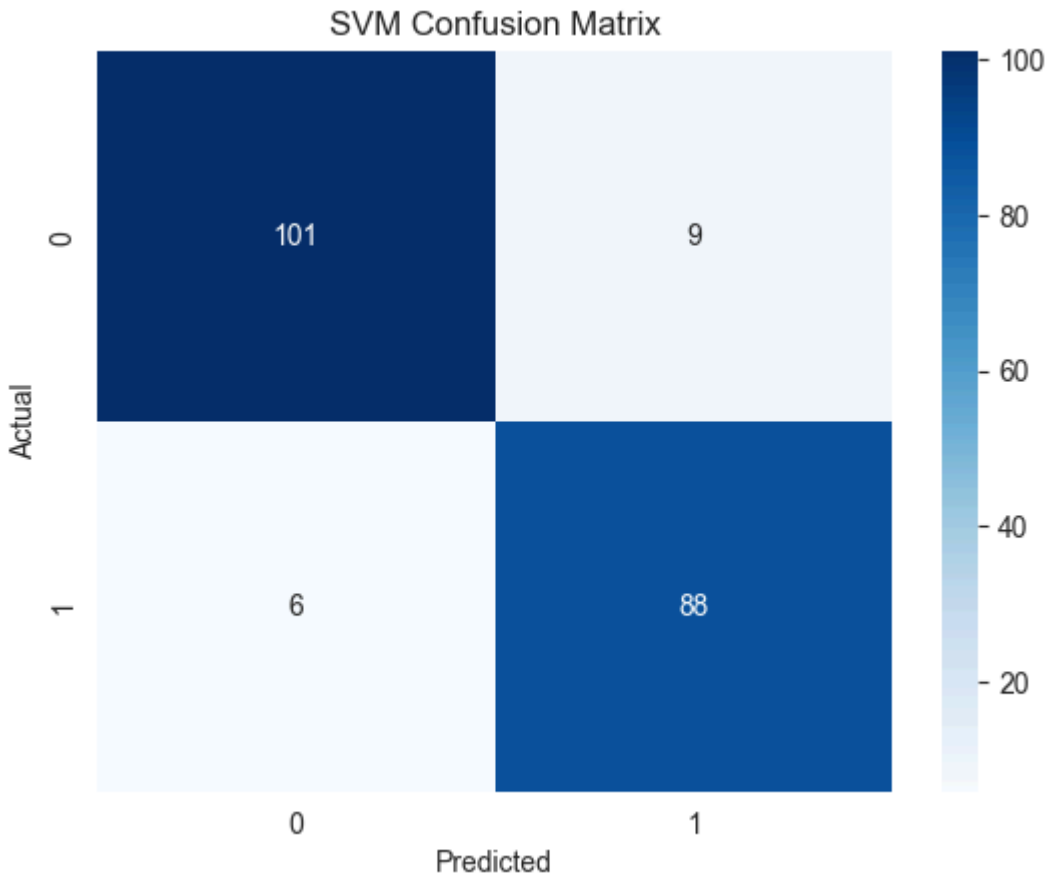
MODEL: Naive Bayes

	precision	recall	f1-score	support
0	0.89	0.81	0.85	110
1	0.80	0.88	0.84	94
accuracy			0.84	204
macro avg	0.84	0.85	0.84	204
weighted avg	0.85	0.84	0.84	204



MODEL: SVM

	precision	recall	f1-score	support
0	0.94	0.92	0.93	110
1	0.91	0.94	0.92	94
accuracy			0.93	204
macro avg	0.93	0.93	0.93	204
weighted avg	0.93	0.93	0.93	204



21 Compare models and identify the winner

In [24]:
```
ranked = results_df.sort_values(
    by=["MCC", "F1", "AUC"],
    ascending=False
).reset_index(drop=True)

display(ranked.round(4))

winner = ranked.loc[0, "Model"]
print("Best model:", winner)
```

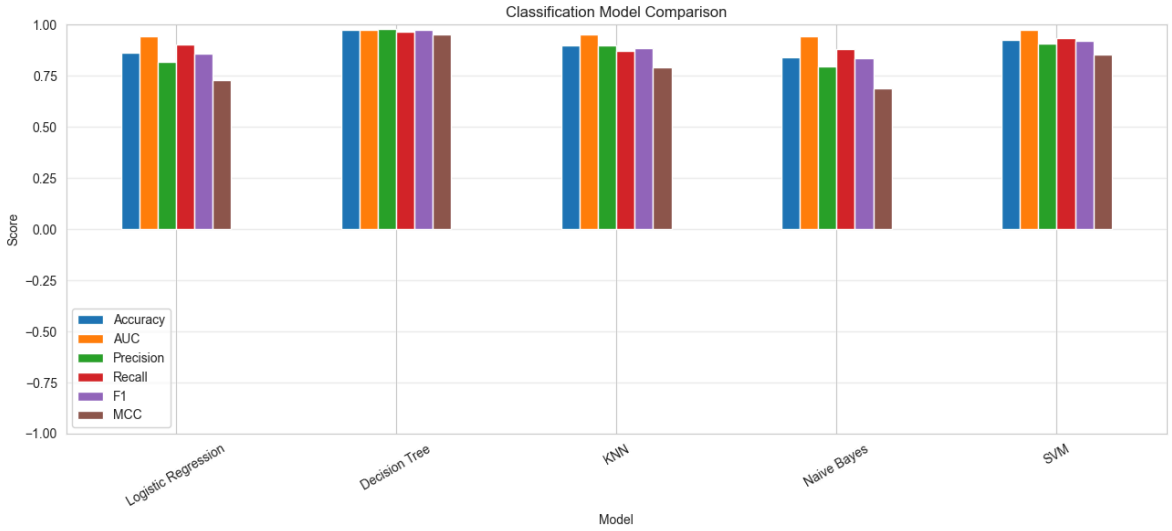
	Model	Accuracy	AUC	Precision	Recall	F1	MCC
0	Decision Tree	0.9755	0.9750	0.9785	0.9681	0.9733	0.9507
1	SVM	0.9265	0.9732	0.9072	0.9362	0.9215	0.8527
2	KNN	0.8971	0.9515	0.9011	0.8723	0.8865	0.7927
3	Logistic Regression	0.8627	0.9434	0.8173	0.9043	0.8586	0.7294
4	Naive Bayes	0.8431	0.9435	0.7981	0.8830	0.8384	0.6901

```
Best model: Decision Tree
```

In [25]:
```
results_df.set_index("Model").plot(
    kind="bar",
    figsize=(13, 6)
)

plt.title("Classification Model Comparison")
```

```
plt.ylabel("Score")
plt.ylim(-1, 1)
plt.xticks(rotation=30)
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()
```



22. Save the trained models and create test data

In [ ]:
```
import joblib

model_dir = Path(r"C:\Users\amanc\OneDrive\Desktop\Heart_Disease_ML_Assignment\M
model_dir.mkdir(exist_ok=True)

file_names = {
    "Logistic Regression": "logistic_regression.pkl",
    "Decision Tree": "decision_tree.pkl",
    "KNN": "knn.pkl",
    "Naive Bayes": "naive_bayes.pkl",
    "SVM": "svm.pkl"
}

for name, model in models.items():
    joblib.dump(model, model_dir / file_names[name])

# Save the held-out test data, including true labels
test_data = X_test.copy()
test_data["target"] = y_test

test_data.to_csv("C:\\Users\\amanc\\OneDrive\\Desktop\\Heart_Disease_ML_Assignme

print("Saved all models and test_data.csv")
```

```
Saved all models and test_data.csv
```

17. Model Training, Evaluation and Comparison – Conclusion

Five machine learning classification algorithms were successfully trained and evaluated on the Heart Disease dataset: Logistic Regression, Decision Tree, K-Nearest Neighbors

(KNN), Gaussian Naive Bayes and Support Vector Machine (SVM).

The dataset was divided into training and testing sets using a group-aware split to prevent identical feature records from appearing in both sets. The training set contained 821 records, while the testing set contained 204 records. Missing-value imputation and feature scaling were included in the model pipelines.

Model Performance Comparison

Model	Accuracy	AUC	Precision	Recall	F1-score	MCC
Logistic Regression	0.8627	0.9434	0.8173	0.9043	0.8586	0.7294
Decision Tree	**0.9755**	**0.9750**	**0.9785**	**0.9681**	**0.9733**	**0.9507**
KNN	0.8971	0.9515	0.9011	0.8723	0.8865	0.7927
Naive Bayes	0.8431	0.9435	0.7981	0.8830	0.8384	0.6901
SVM	0.9265	0.9732	0.9072	0.9362	0.9215	0.8527

Key Observations

1. **Decision Tree achieved the best overall performance**, with an accuracy of 97.55%, F1-score of 0.9733 and MCC of 0.9507.
2. SVM was the second-best model, achieving 92.65% accuracy and an F1-score of 0.9215.
3. KNN achieved 89.71% accuracy, demonstrating reasonably good classification performance.
4. Logistic Regression achieved 86.27% accuracy, while Gaussian Naive Bayes recorded the lowest accuracy of 84.31%.
5. The Decision Tree confusion matrix showed 108 true negatives, 91 true positives, 2 false positives and 3 false negatives, resulting in only five incorrect predictions out of 204 test records.

Best Model Selection

Based on the evaluation metrics, **Decision Tree was selected as the best-performing classification model** for this experiment.

It achieved the highest Accuracy, AUC, Precision, Recall, F1-score and MCC among the five models tested. Its high MCC indicates strong agreement between predicted and actual labels on the held-out test dataset.

Final Conclusion

The experiment successfully implemented and compared five classification algorithms on the same Heart Disease dataset. All six required evaluation metrics were calculated, and

the results demonstrated that the Decision Tree classifier performed best on the selected test partition.

The trained models were saved as `.pkl` files for integration into the Streamlit application, allowing users to select different models and compare their predictions and evaluation results.

Although the Decision Tree achieved excellent performance, these results are limited to the available dataset and the chosen train-test split. Additional cross-validation and testing on independent data would be necessary to assess generalization and clinical reliability.

Overall, the project demonstrates the complete machine learning workflow, including data preprocessing, exploratory analysis, model training, evaluation, comparison and deployment preparation.